

Kreacijski paterni

Singleton patern

Singleton patern koristi se kada je potrebno obezbijediti da u sistemu postoji samo jedna instanca određene klase. U našem sistemu, ovaj patern bio bi veoma koristan za upravljanje konekcijom prema bazi podataka ili za centralizovano upravljanje evidencijom sistema.

Tokom rada, veliki broj klasa pristupa podacima (korisnici, knjige, posudbe,...). Kada bi svaka klasa pravila svoju konekciju prema bazi podataka, došlo bi do nepotrebnog trošenja memorije i resursa sistema. Singleton patern omogućava da sve klase koriste jednu zajedničku instancu klase za pristup podacima.

Na primjer, kada bibliotekar odobri zahtjev za posudbu, sistem mora promijeniti status knjige, kreirati novu posudbu i evidentirati korisnika koji je izvršio posudbu. Sve ove operacije mogu koristiti jednu zajedničku instancu klase za rad sa bazom podataka.

Factory Method patern

Factory Method patern koristi se za centralizovano kreiranje objekata i odvajanje logike kreiranja od ostatka sistema. U našem sistemu ovaj patern mogao bi se koristiti za kreiranje korisnika sistema.

Klasa *KorisnikSistema* sadrži atribut *Uloga*, pri čemu korisnik može imati jednu od uloga: *clan*, *bibliotekar* i *administrator*. Iako sistem već zna ulogu koju korisnik ima, Factory Method patern omogućava da se kompletna logika kreiranja korisnika nalazi na jednom mjestu. Umjesto direktnog kreiranja objekata kroz različite dijelove sistema, mogla bi postojati posebna klasa *UserFactory* koja bi kreirala korisnike na osnovu njihove uloge.

Abstract Factory patern

Abstract Factory patern koristi se za kreiranje grupa međusobno povezanih objekata. U našem sistemu ovaj patern bio bi koristan tokom procesa posudbe knjiga. Kada korisnik želi posuditi knjigu, sistem mora izvršiti više povezanih operacija (kreirati zahtjev za posudbu, kreirati posudbu, provjeriti validnost članske kartice i promijeniti status knjige).

Umjesto da se svi ovi objekti kreiraju zasebno u različitim dijelovima aplikacije, Abstract Factory omogućava da se kompletan proces kreiranja obavlja preko jedne centralizovane fabrike. Klasa *BibliotekaFactory* mogla bi automatski kreirati objekat zahtjeva za posudbu, kreirati objekat posudbe, postaviti odgovarajući status knjige i izvršiti provjeru članstva korisnika. Na taj način svi povezani objekti nastaju organizovano i kontrolisano.

Builder patern

Builder patern koristi se za kreiranje složenih objekata koji imaju veliki broj atributa i različitih koraka kreiranja. U našem sistemu ovaj patern bi bio koristan za klasu *NabavkaKnjiga*. Objekat nabavke knjiga sadrži veliki broj podataka: knjigu, dobavljača, količinu, datum narudžbe, očekivani datum isporuke i status nabavke.

Builder patern omogućava postepeno kreiranje objekta korak po korak. Proces kreiranja mogao bi izgledati:

1. Bira se dobavljač,
2. Dodaje se knjiga,
3. Određuje količina,
4. Postavlja datum isporuke,
5. Kreira konačna narudžba.

Na ovaj način kod postaje pregledniji i jednostavniji za održavanje.

Prototype patern

Prototype patern koristi se za kreiranje novih objekata kopiranjem postojećih objekata. U našem sistemu ovaj patern mogao bi se koristiti kod kreiranja novih primjeraka knjiga. Biblioteka posjeduje više primjeraka iste knjige, pri čemu veliki broj podataka ostaje isti (naziv knjige, autor, kategorija).

Umjesto ponovnog unosa svih podataka za svaki primjerak knjige, sistem može kopirati postojeći objekat knjige i promijeniti samo određene informacije, kao što je identifikacioni broj primjerka. Na taj način ubrzava se rad sistema i smanjuje mogućnost greške prilikom unosa podataka.